VPC Service: Zonal NAT Gateway

VPC Service

26th October 2026

Overview

This document outlines the step-by-step practical implementation of a foundational AWS networking architecture. The configuration includes the creation of a custom Virtual Private Cloud (VPC), a Public Subnet, a Private Subnet, and the creation and attachment of an Internet Gateway (IGW). This setup serves as the prerequisite infrastructure for deploying a NAT Gateway.

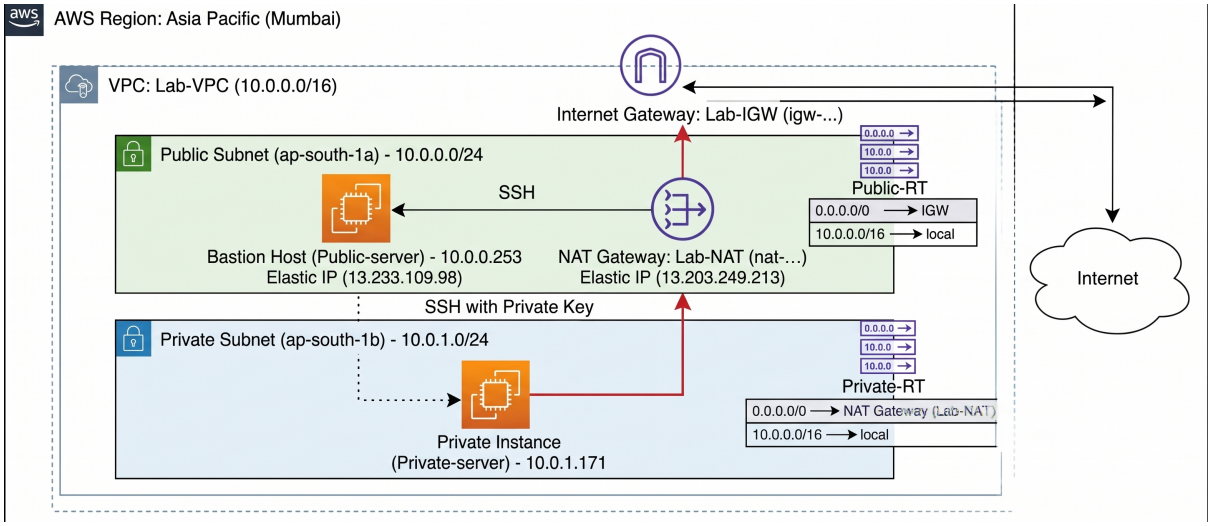


Figure 1: AWS network topology

1 Creating the Virtual Private Cloud (VPC)

1.1 Initiating VPC Creation

Navigate to the VPC Dashboard in the AWS Management Console and click on the **Create VPC** button.

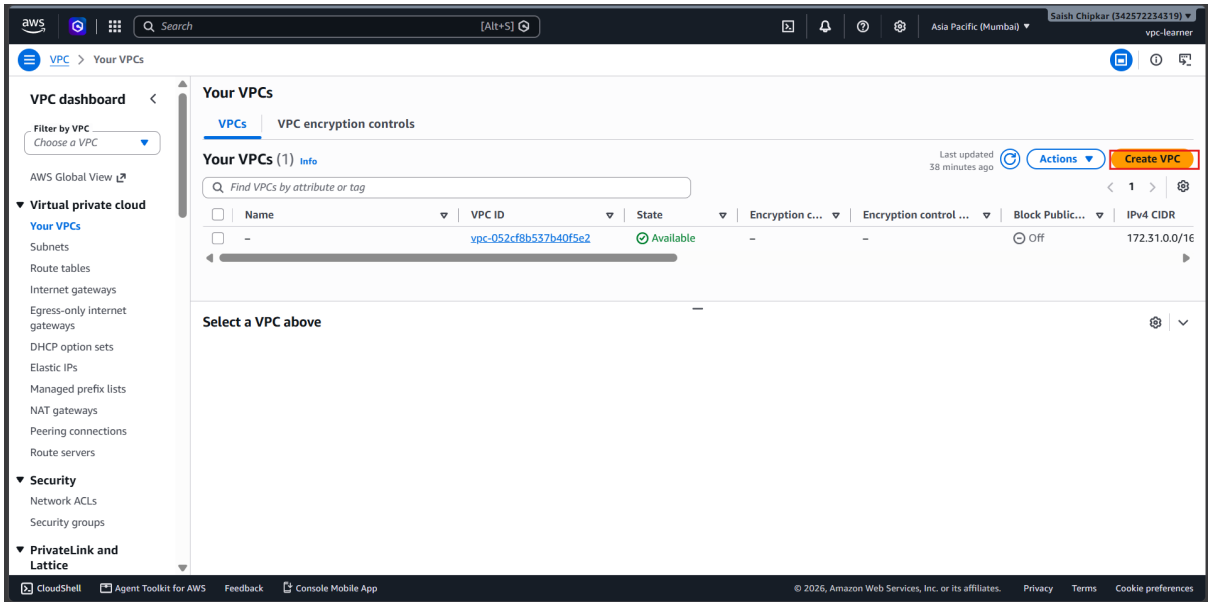


Figure 2: VPC Dashboard - Create VPC

1.2 Configuring VPC Details

Select "VPC only". Assign the Name tag as `Lab-VPC` and configure the IPv4 CIDR block manually to `10.0.0.0/16`.

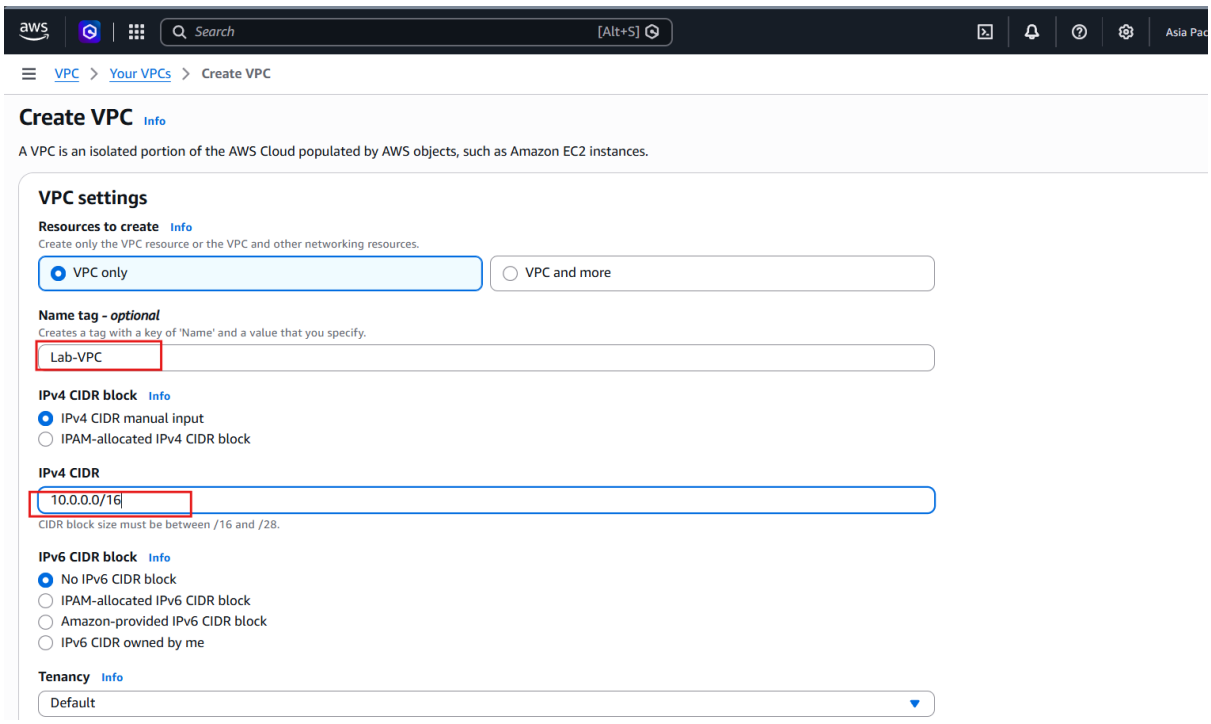


Figure 3: Setting VPC Name and CIDR block

Leave the rest of the settings (like IPv6 CIDR and Tenancy) to their defaults and click **Create VPC** at the bottom of the page.

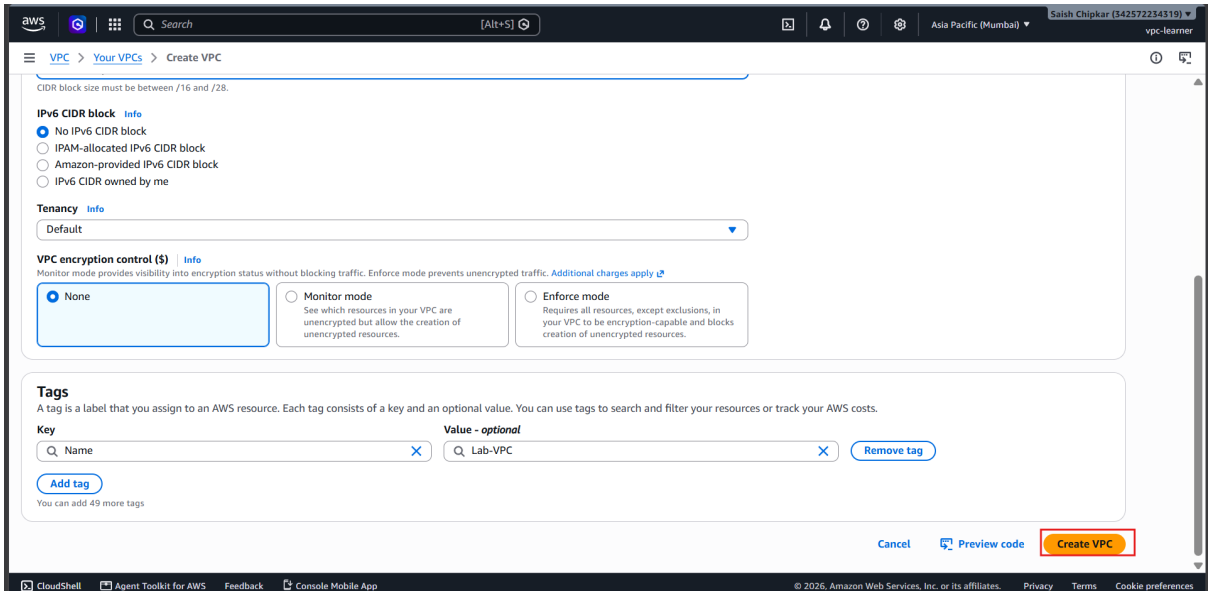


Figure 4: Confirming VPC Creation

The console will display a success message confirming the creation of `Lab-VPC` along with its unique VPC ID.

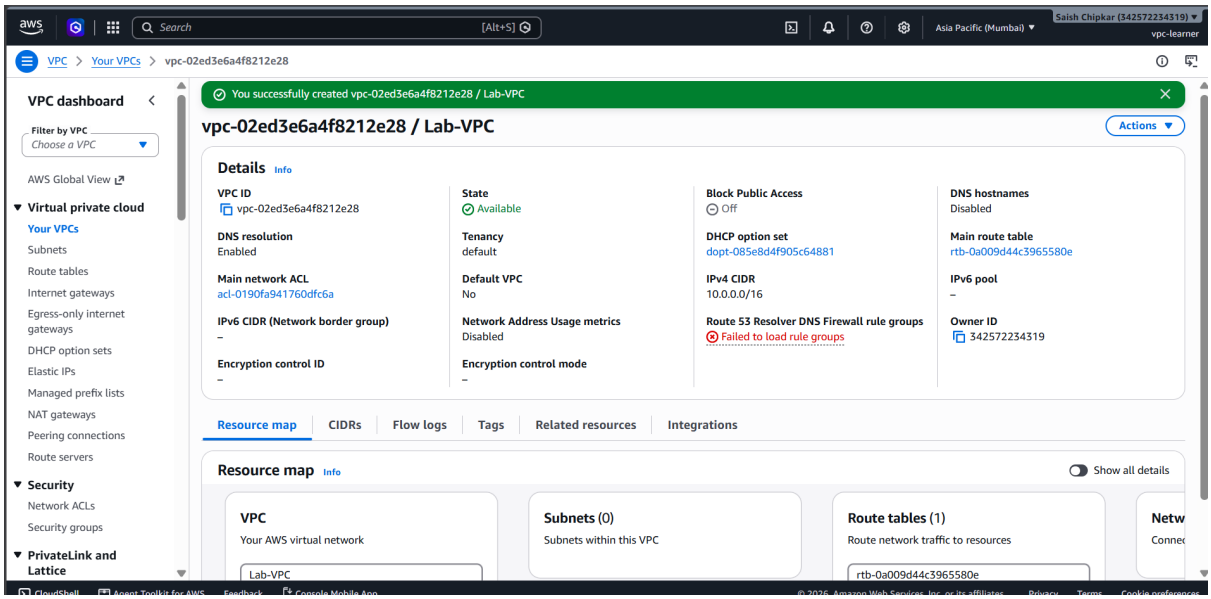


Figure 5: Successful VPC Creation Screen

2 Creating Subnets

2.1 Initiating Subnet Creation

Navigate to the **Subnets** menu on the left sidebar and click on **Create subnet**.

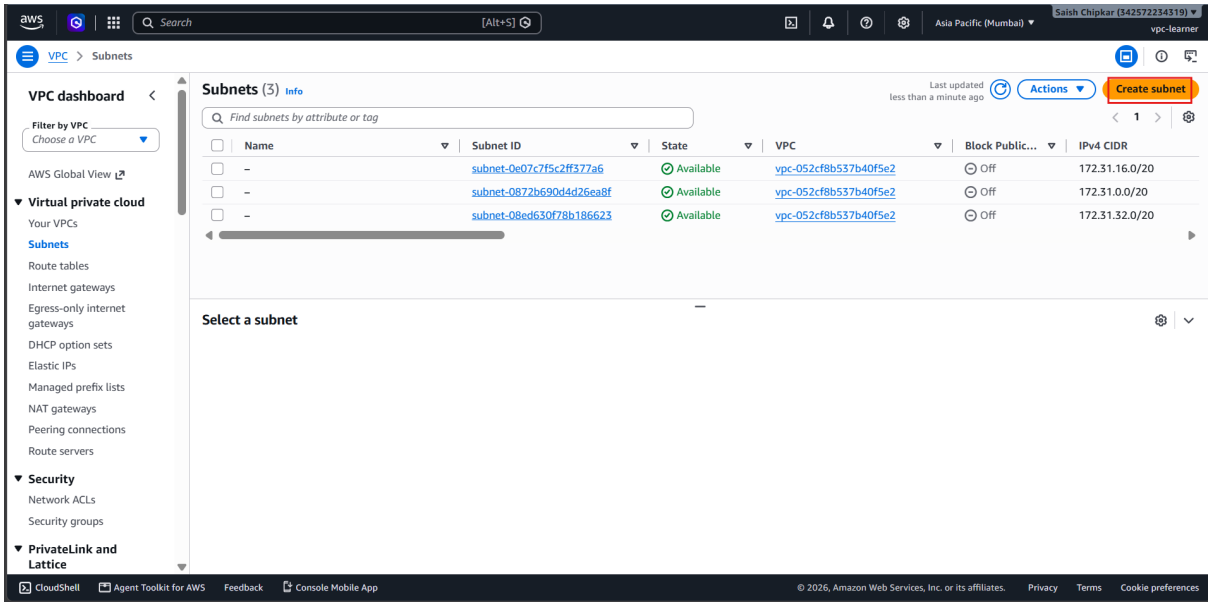


Figure 6: Subnets Dashboard

2.2 Configuring the Public Subnet

Select the previously created `Lab-VPC` from the dropdown. Name the subnet `Public-Subnet` and assign it to the Availability Zone `ap-south-1a`.

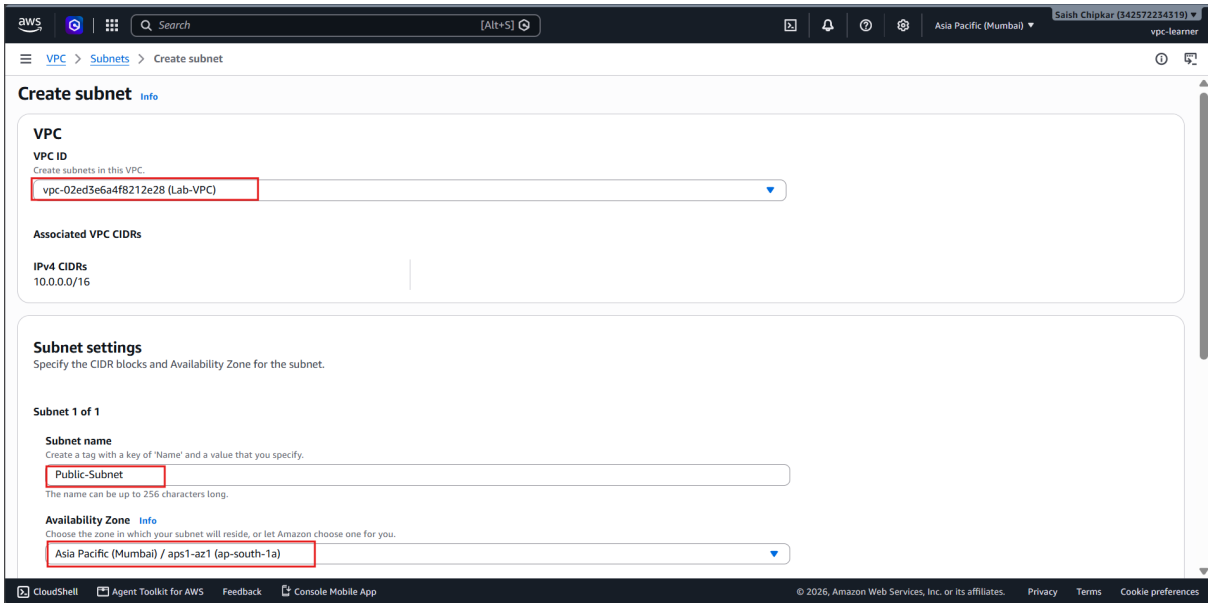


Figure 7: Selecting VPC and naming the Public Subnet

Assign the IPv4 subnet CIDR block as `10.0.0.0/24` and click **Create subnet**.

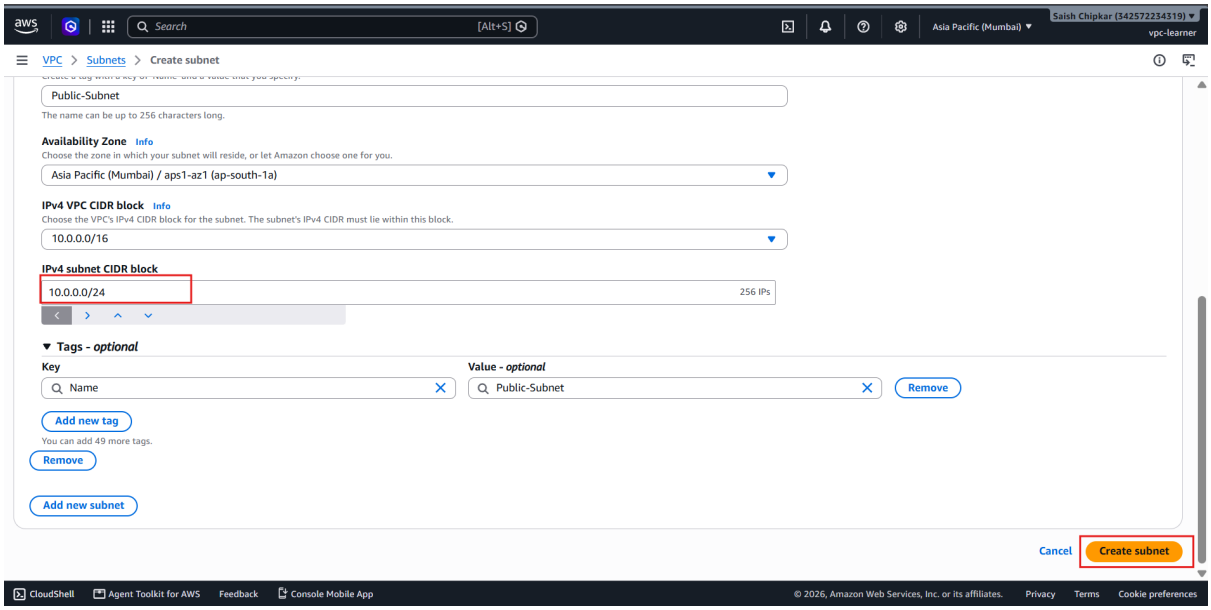


Figure 8: Setting the Public Subnet CIDR block

2.3 Configuring the Private Subnet

Repeat the subnet creation process for the private tier. Name the subnet `Private-Subnet`, place it in a different Availability Zone (`ap-south-1b`), and set its IPv4 CIDR block to `10.0.1.0/24`. Click **Create subnet**.

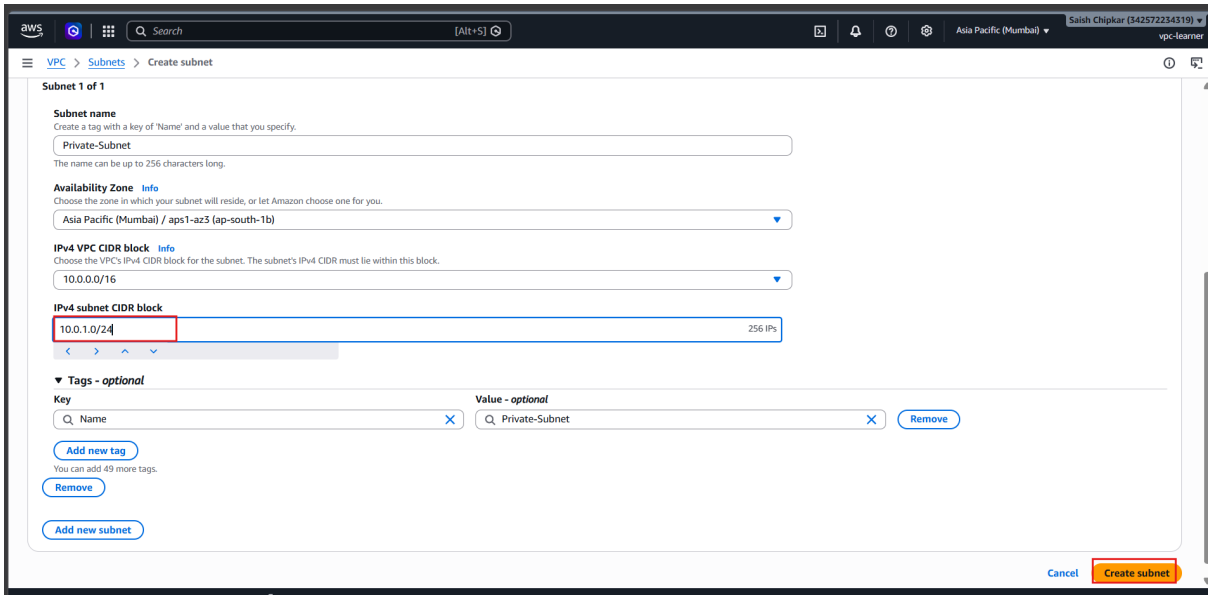


Figure 9: Creating the Private Subnet

3 Creating and Attaching the Internet Gateway

3.1 Creating the IGW

Navigate to **Internet gateways** in the sidebar and create a new gateway. Assign the Name tag `Lab-IGW` and click **Create internet gateway**.

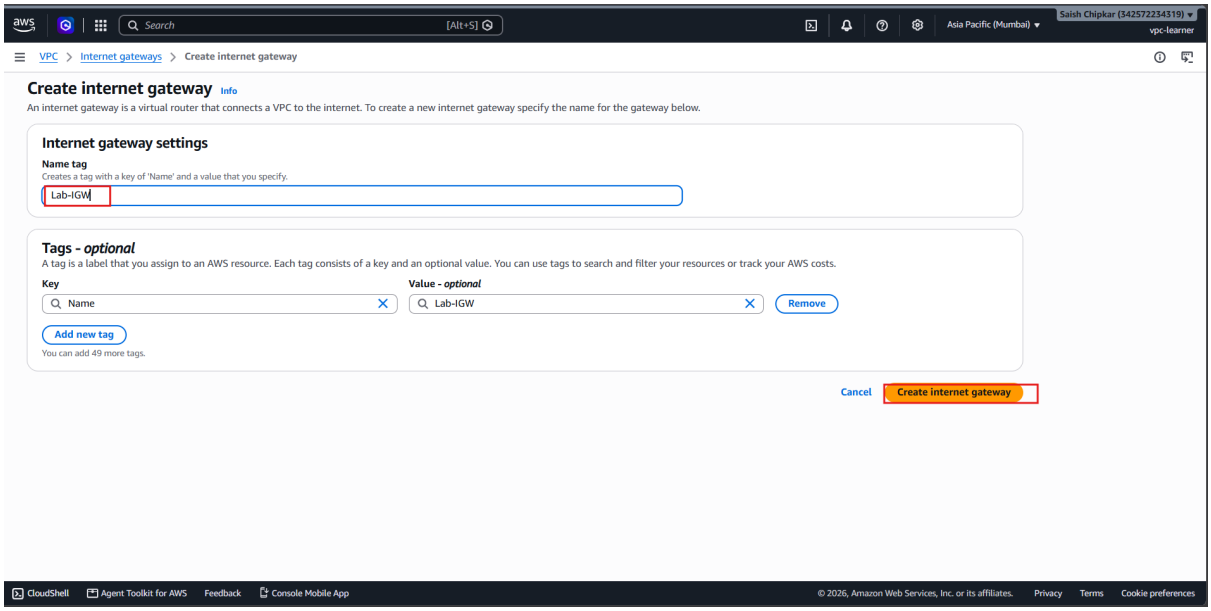


Figure 10: Creating the Internet Gateway

3.2 Attaching the IGW to the VPC

Once created, the IGW will be in a "Detached" state. Choose the action to attach it to a VPC, select `Lab-VPC` from the dropdown list, and click **Attach internet gateway**. This allows resources in the public subnet to route traffic to the external internet.

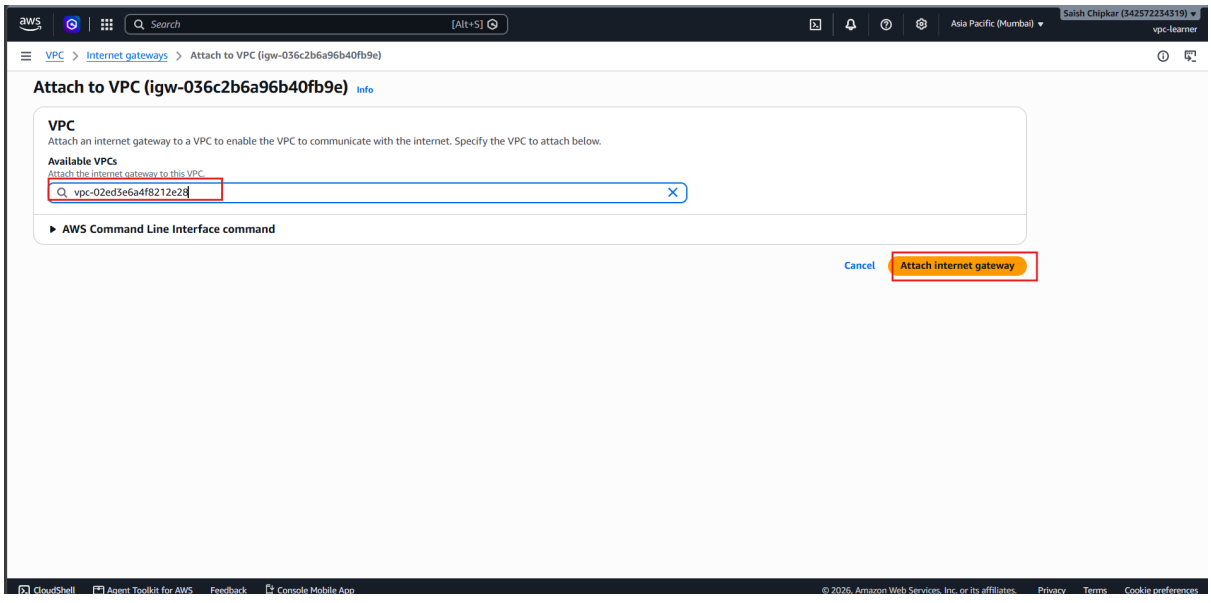


Figure 11: Attaching the Internet Gateway to Lab-VPC

4 Creating the Virtual Private Cloud (VPC)

4.1 Initiating VPC Creation

Navigate to the VPC Dashboard in the AWS Management Console and click on the **Create VPC** button.

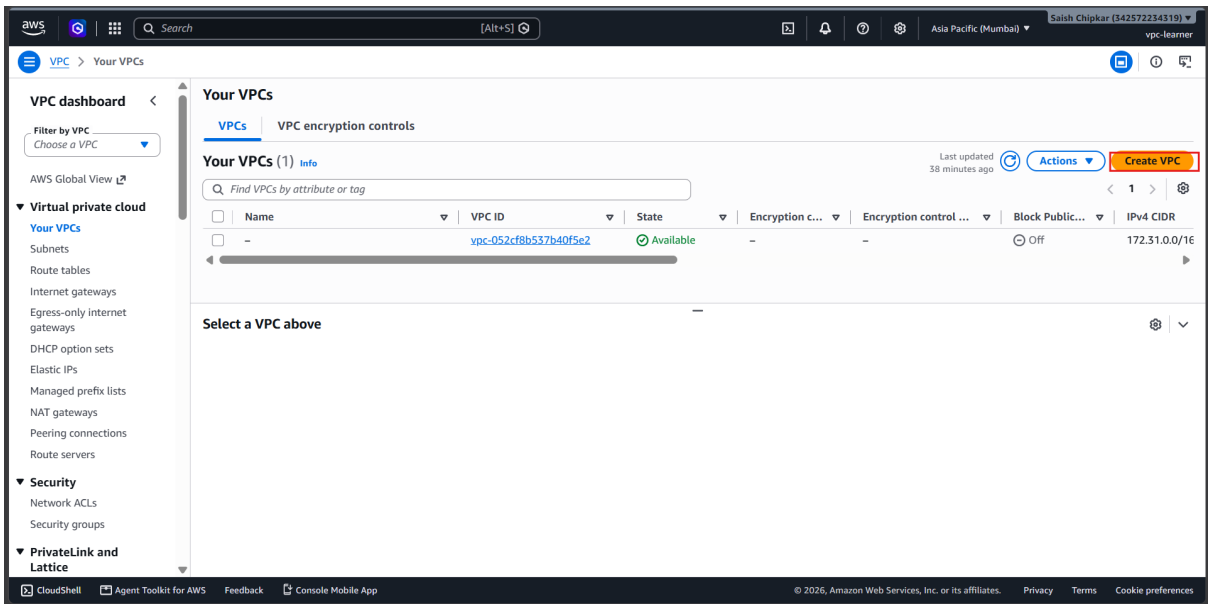


Figure 12: VPC Dashboard - Create VPC

4.2 Configuring VPC Details

Select "VPC only". Assign the Name tag as `Lab-VPC` and configure the IPv4 CIDR block manually to `10.0.0.0/16`.

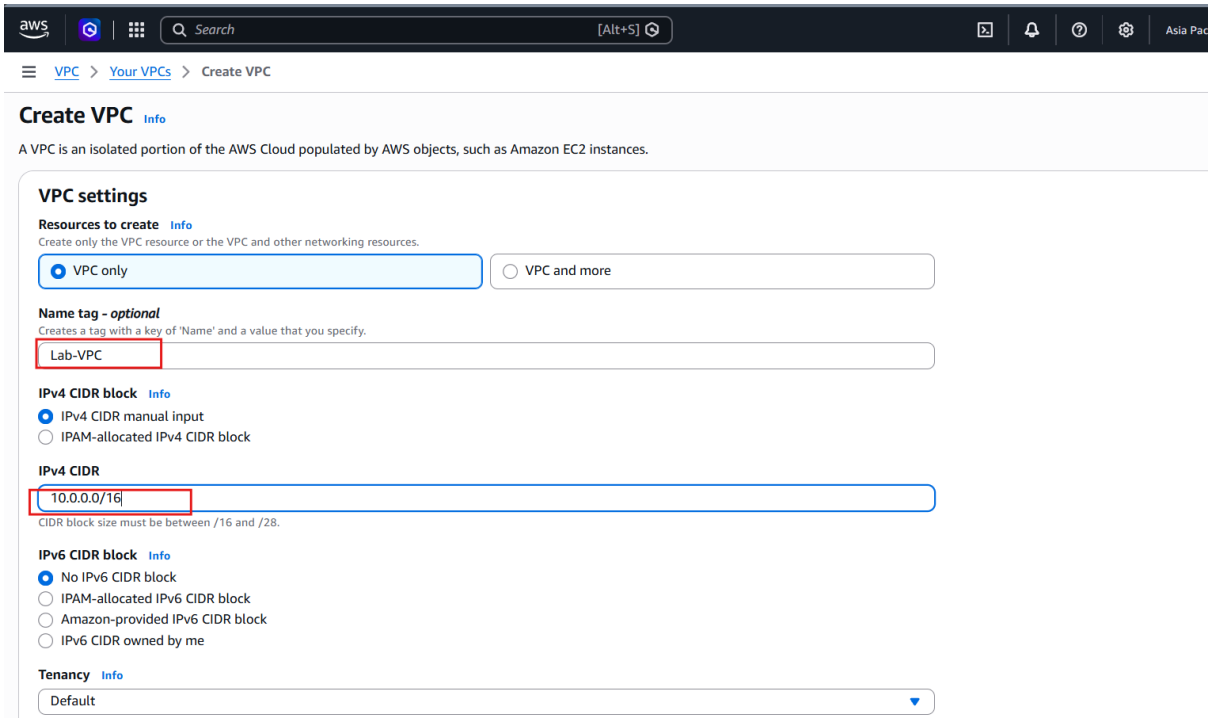


Figure 13: Setting VPC Name and CIDR block

Leave the rest of the settings to their defaults and click **Create VPC** at the bottom of the page.

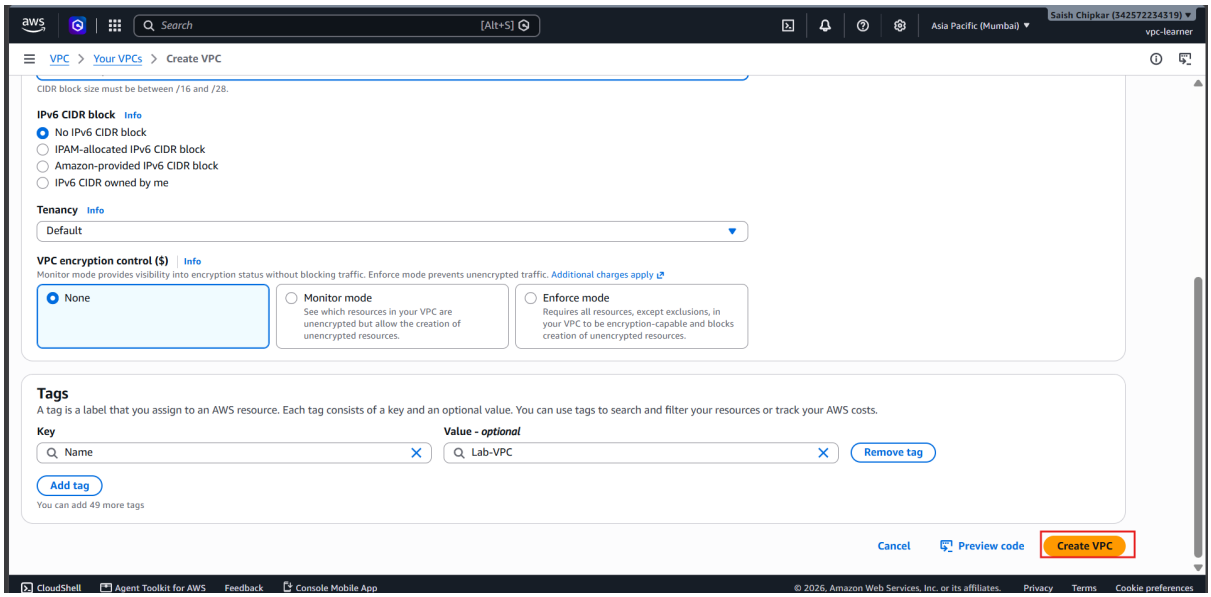


Figure 14: Confirming VPC Creation

The console will display a success message confirming the creation of `Lab-VPC` along with its unique VPC ID.

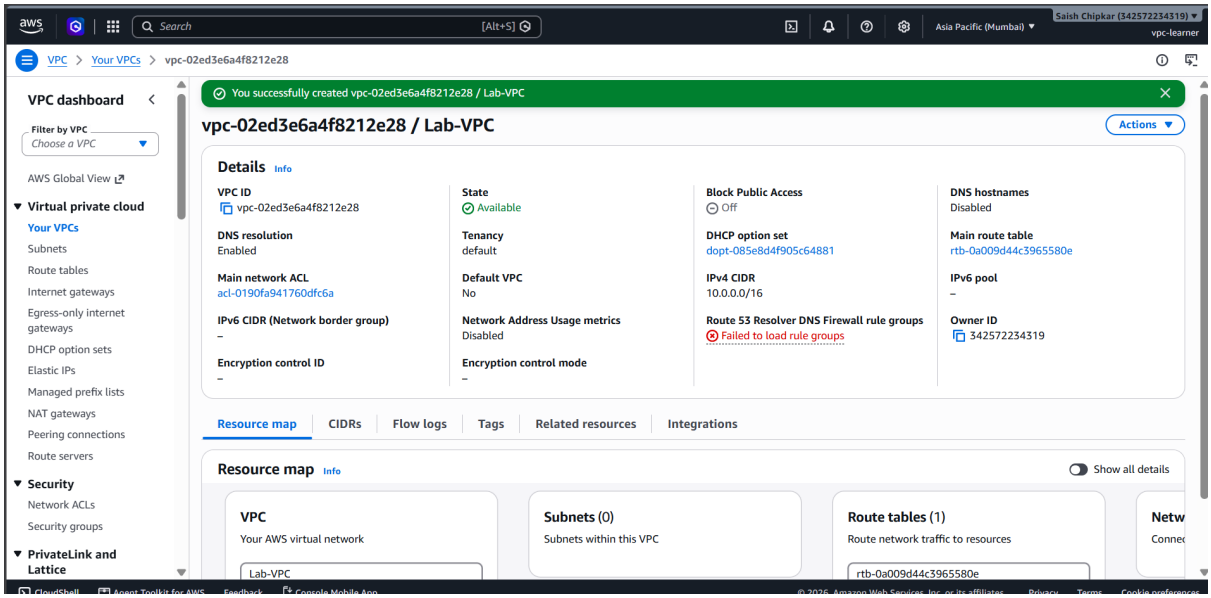


Figure 15: Successful VPC Creation Screen

5 Creating Subnets

5.1 Initiating Subnet Creation

Navigate to the **Subnets** menu on the left sidebar and click on **Create subnet**.

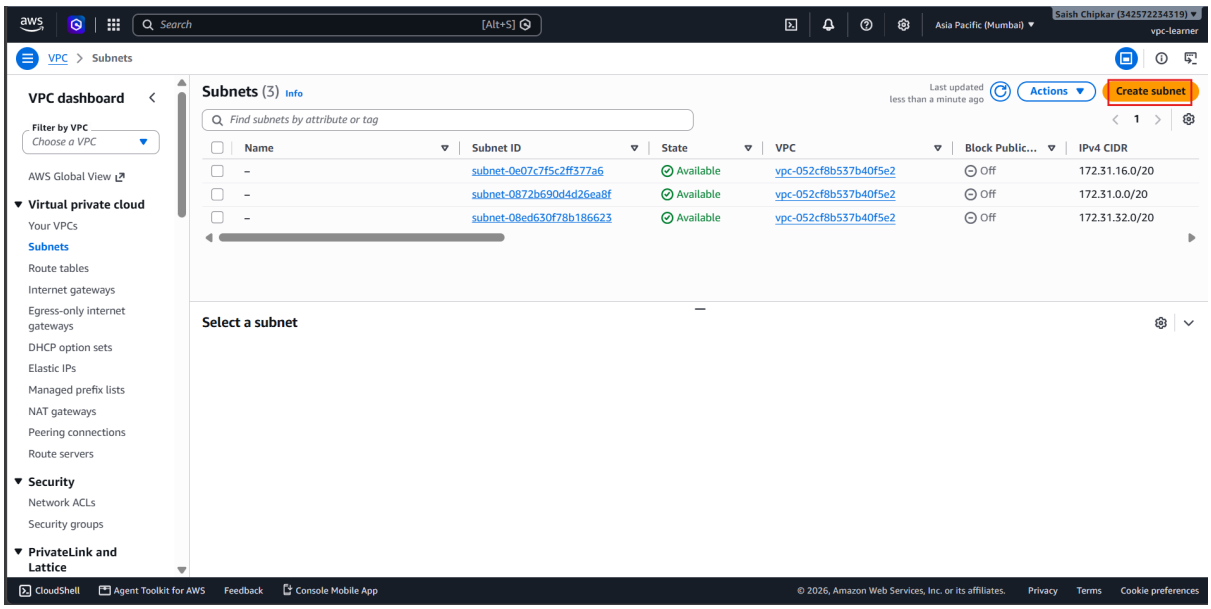


Figure 16: Subnets Dashboard

5.2 Configuring the Public Subnet

Select the previously created `Lab-VPC` from the dropdown. Name the subnet `Public-Subnet` and assign it to the Availability Zone `ap-south-1a`.

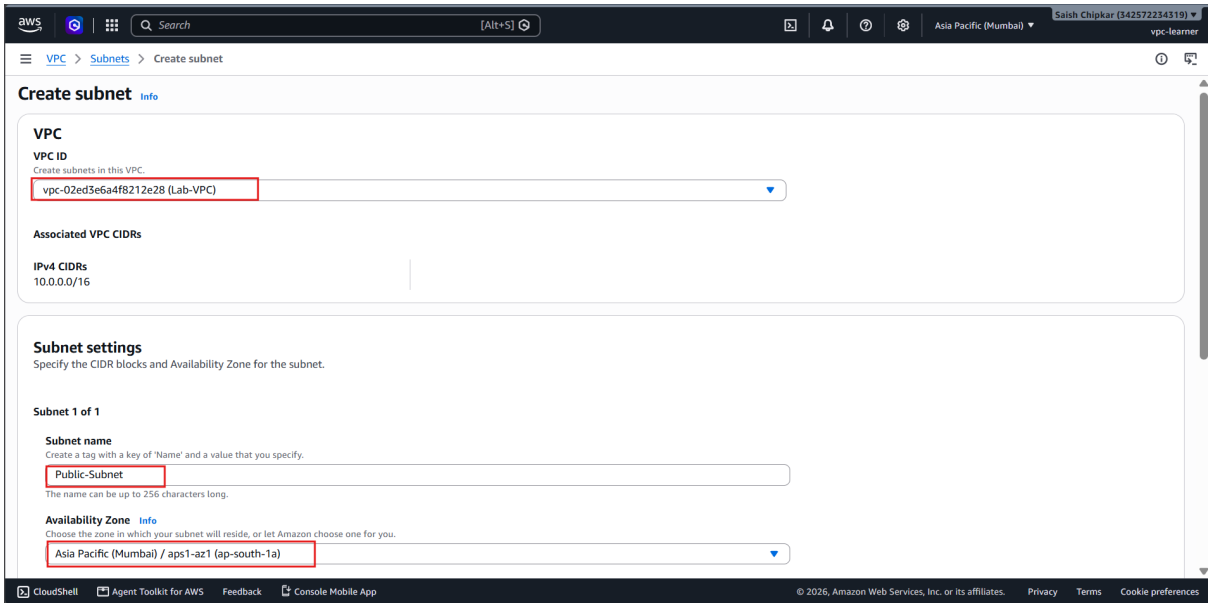


Figure 17: Selecting VPC and naming the Public Subnet

Assign the IPv4 subnet CIDR block as `10.0.0.0/24` and click **Create subnet**.

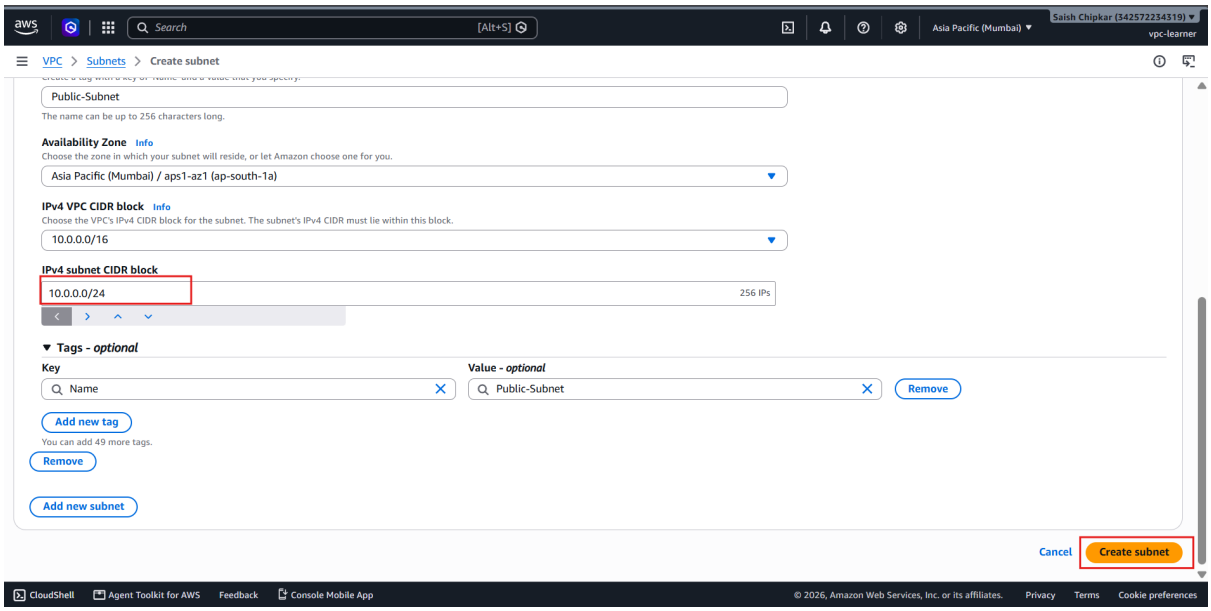


Figure 18: Setting the Public Subnet CIDR block

5.3 Configuring the Private Subnet

Repeat the subnet creation process for the private tier. Name the subnet `Private-Subnet`, place it in a different Availability Zone (`ap-south-1b`), and set its IPv4 CIDR block to `10.0.1.0/24`. Click **Create subnet**.

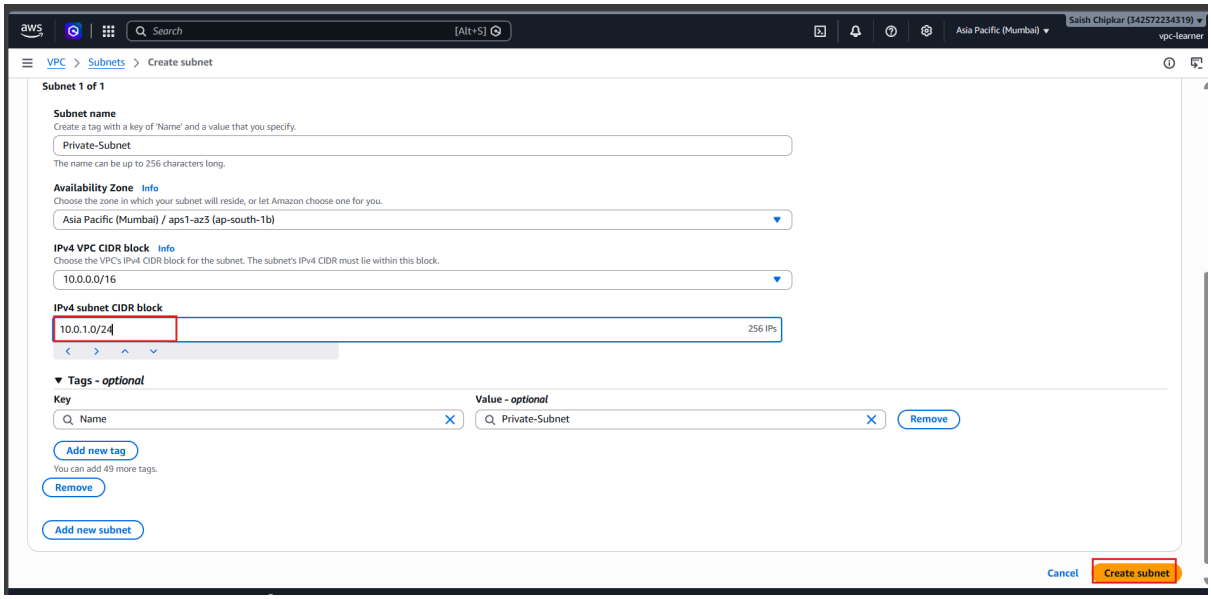


Figure 19: Creating the Private Subnet

6 Creating and Attaching the Internet Gateway

6.1 Creating the IGW

Navigate to **Internet gateways** in the sidebar and create a new gateway. Assign the Name tag `Lab-IGW` and click **Create internet gateway**.

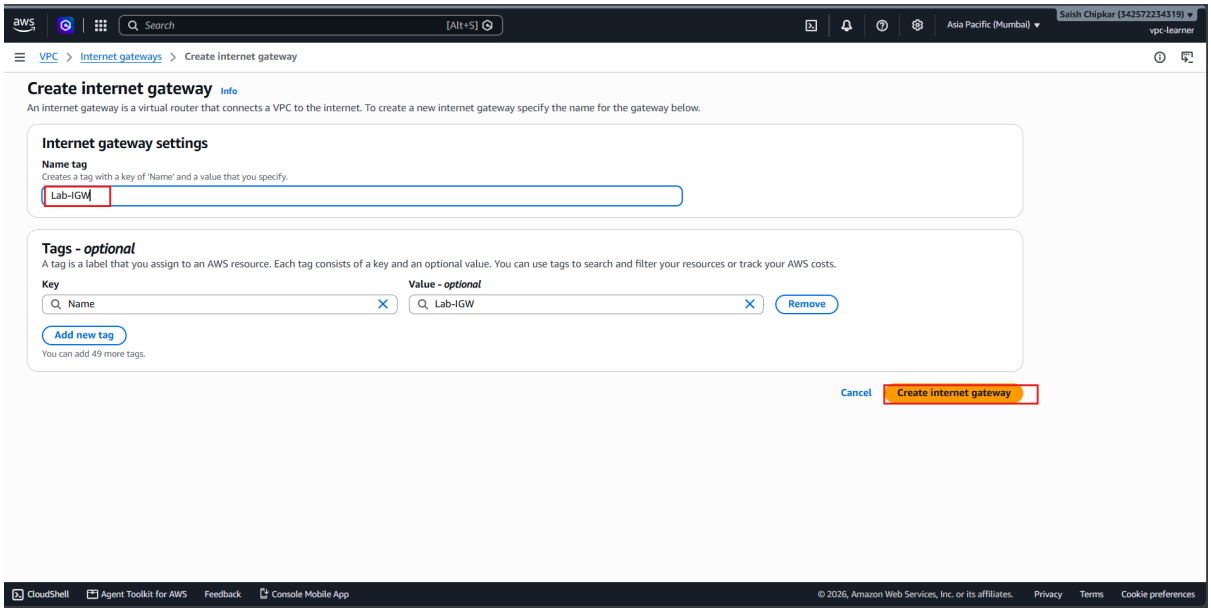


Figure 20: Creating the Internet Gateway

6.2 Attaching the IGW to the VPC

Choose the action to attach it to a VPC, select `Lab-VPC` from the dropdown list, and click **Attach internet gateway**. A success message will confirm the gateway is successfully

attached to the VPC[cite: 10, 11].

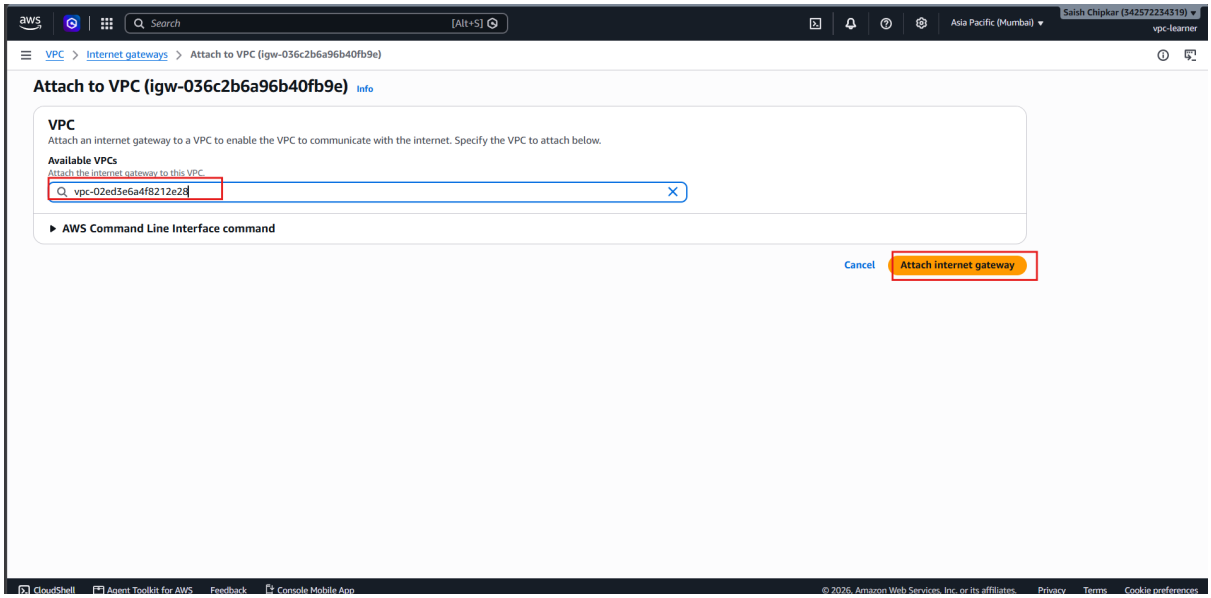


Figure 21: Attaching the Internet Gateway to Lab-VPC

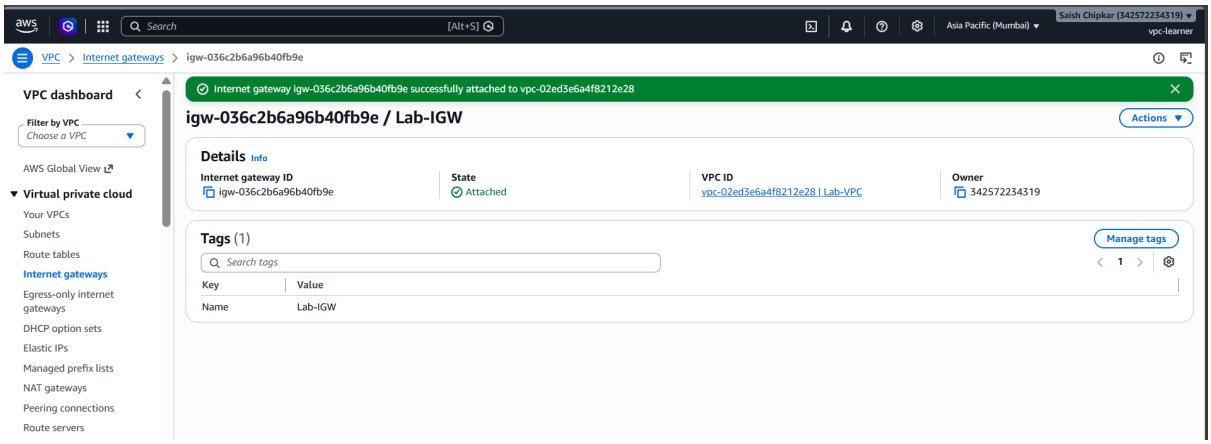


Figure 22: Successful Attachment Confirmation

7 Allocating an Elastic IP

Before creating a public-facing NAT Gateway, an Elastic IP is required. Navigate to **Elastic IPs**, ensure the correct network border group (e.g., `ap-south-1`) is selected, and click **Allocate**[cite: 12]. A success banner will confirm the allocation of the new IPv4 address[cite: 13].

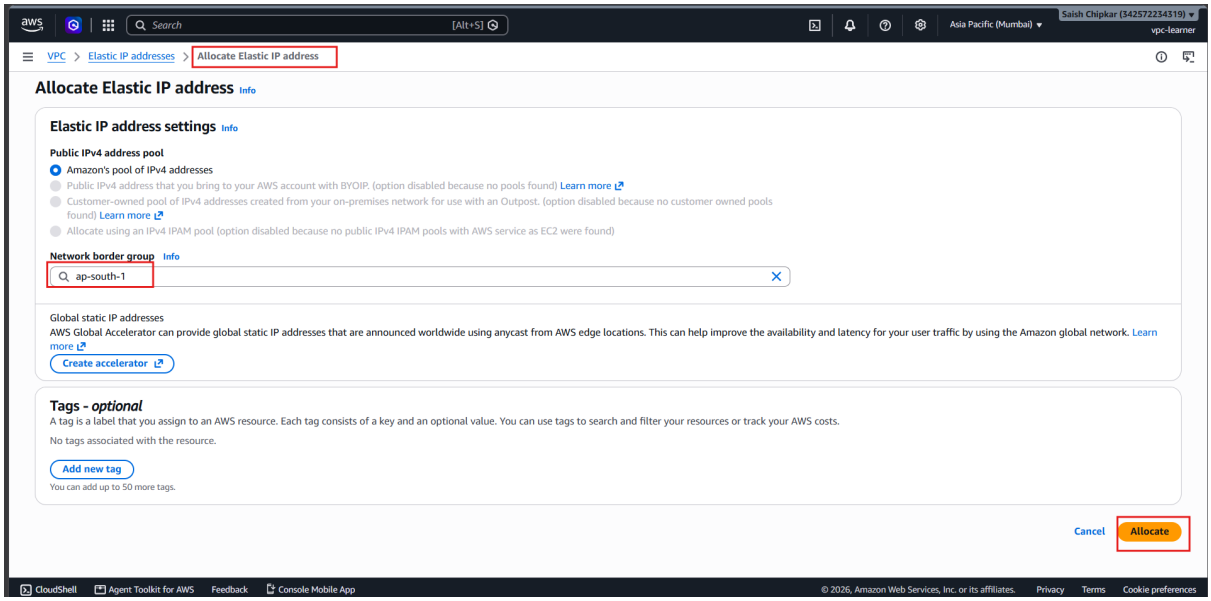


Figure 23: Allocating a new Elastic IP

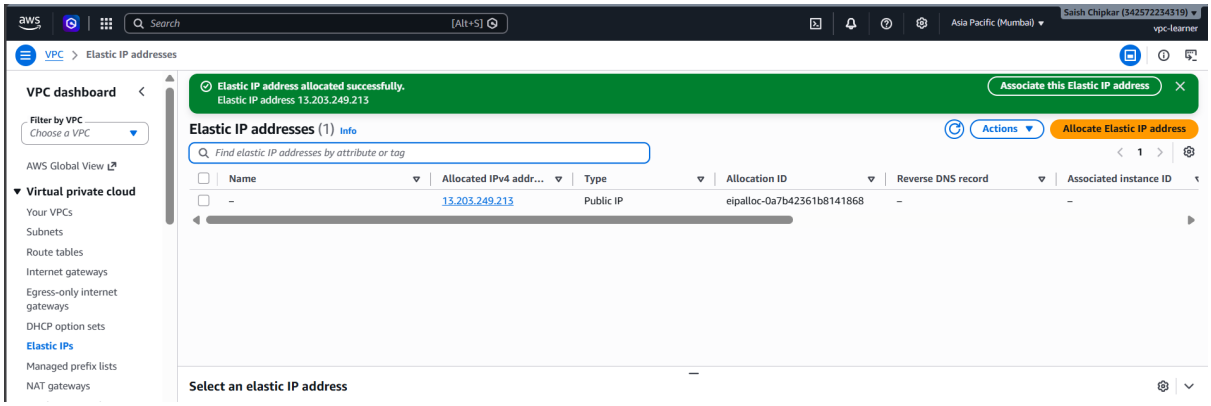


Figure 24: Elastic IP allocated successfully

8 Creating the NAT Gateway

Navigate to **NAT gateways** in the sidebar and click **Create NAT gateway**[cite: 14]. Name the gateway `Lab-NAT`, select **Zonal** availability, and assign it to the `Private-Subnet`[cite: 15]. Associate the previously allocated Elastic IP and click **Create NAT gateway**[cite: 15, 16].

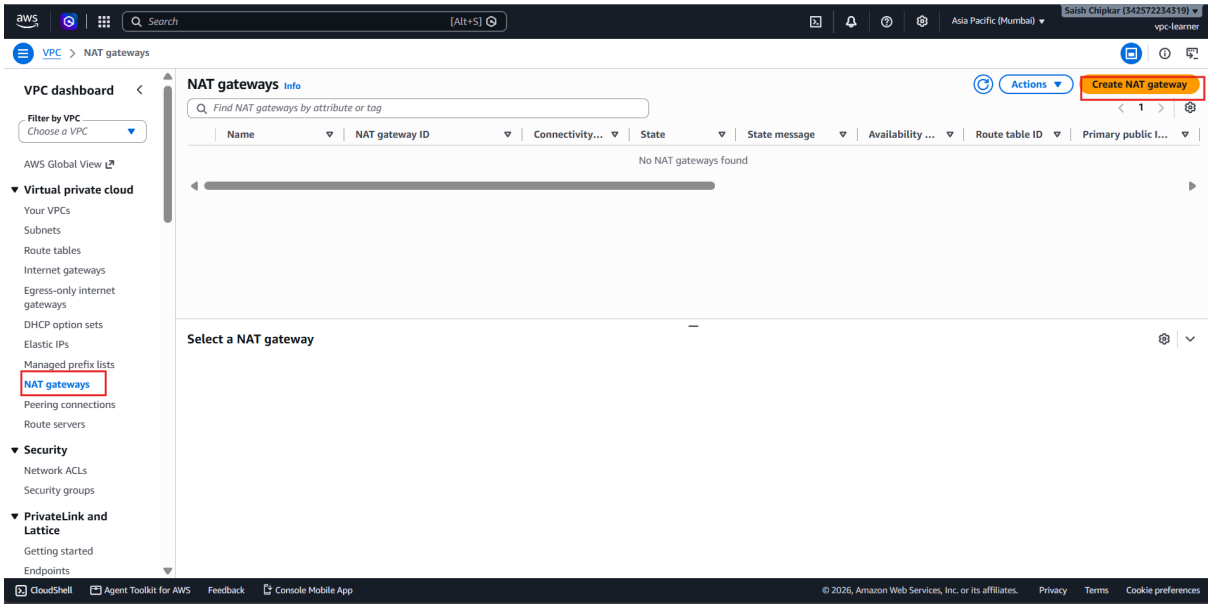


Figure 25: NAT Gateways Dashboard

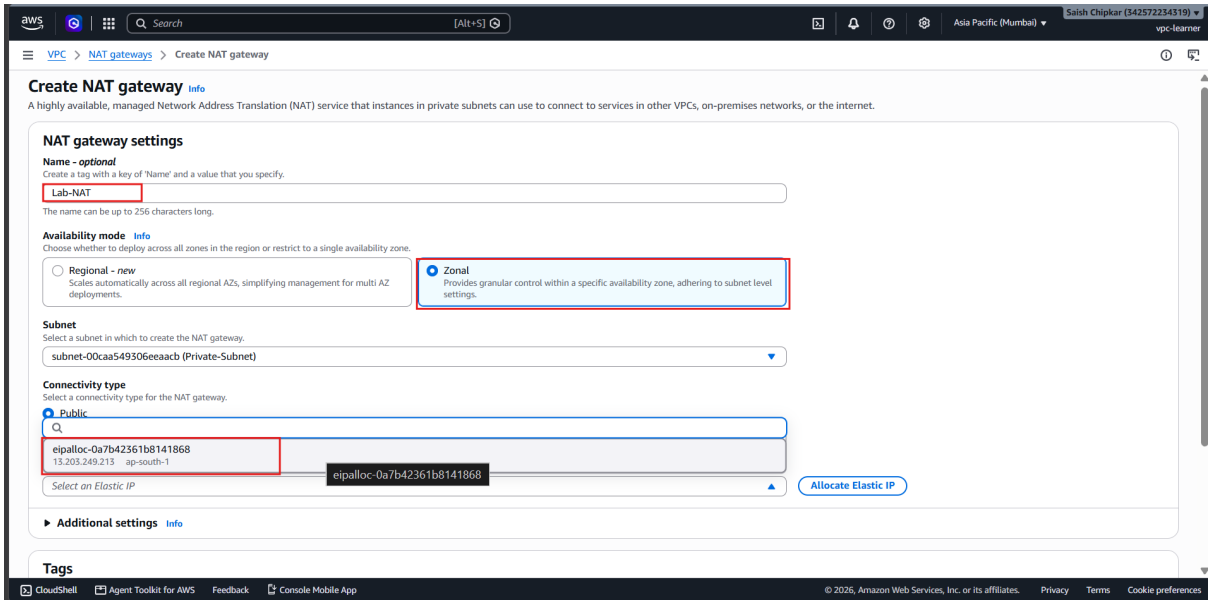


Figure 26: Configuring NAT Gateway Settings

9 Architectural Concepts: Zonal vs. Regional NAT Gateways

When deploying a NAT Gateway, AWS provides two availability modes: Zonal and Regional. Understanding the distinction is critical for cost optimization and high availability.

9.1 Zonal NAT Gateways

A Zonal NAT Gateway is deployed into a specific public subnet within a single Availability Zone (AZ).

While it is technically possible for a private instance in AZ-B to route its outbound internet traffic through a Zonal NAT Gateway located in AZ-A, doing so introduces significant architectural flaws:

- **Cross-AZ Data Charges:** AWS charges a data transfer fee for traffic crossing the physical boundary between Availability Zones. Routing internet traffic from AZ-B to AZ-A incurs these additional costs on top of the standard NAT processing hourly and data processing fees.
- **Single Point of Failure:** If AZ-A experiences an outage, instances in AZ-B will immediately lose their internet connection because their only outbound routing path is located in the degraded zone.

AWS Best Practice for Zonal NAT: To achieve true high availability and avoid Cross-AZ data fees, a separate Zonal NAT Gateway must be deployed in *every* AZ where private subnets exist. Each private subnet's Route Table must then be configured to point strictly to the NAT Gateway residing within its own local AZ.

9.2 Regional NAT Gateways

A Regional NAT Gateway operates at the VPC level rather than the subnet level.

Instead of manually deploying multiple gateways and managing distinct route tables, a single Regional NAT Gateway is provisioned. AWS automatically expands the gateway's underlying infrastructure into new Availability Zones in the background as workloads are detected in them. This provides central management, requires only a single unified route table for all private subnets, and eliminates the risk of accidental Cross-AZ routing fees.

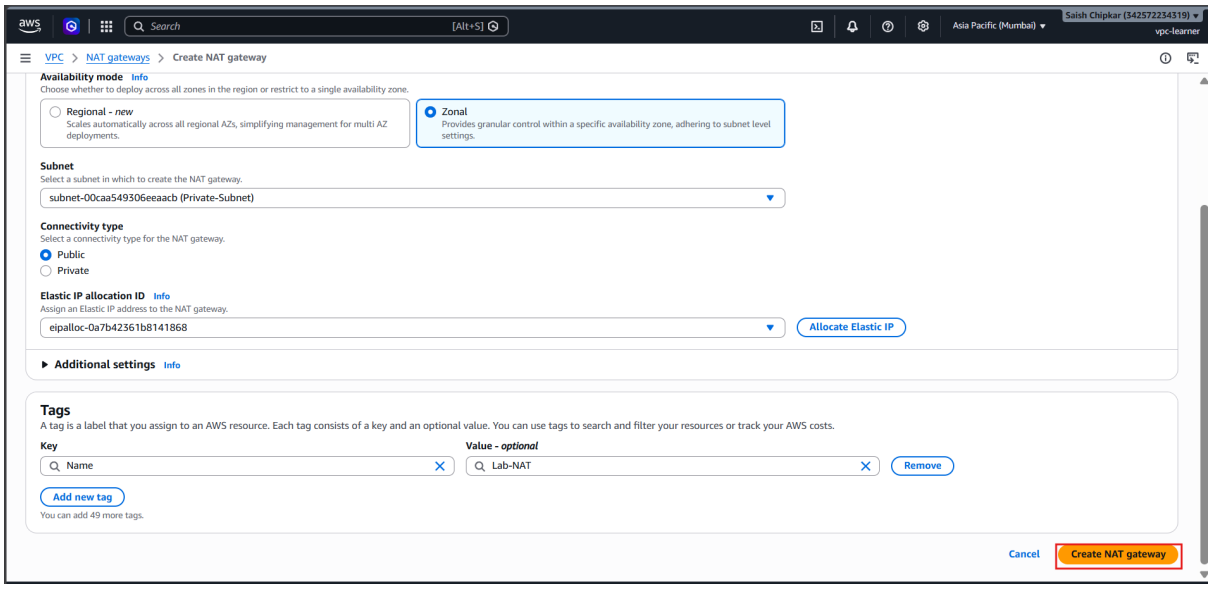


Figure 27: Creating the NAT Gateway

10 Configuring Route Tables

10.1 Public Route Table

Navigate to **Route tables** and click **Create route table**. Name it `Public-RT` and associate it with `Lab-VPC`[cite: 17].

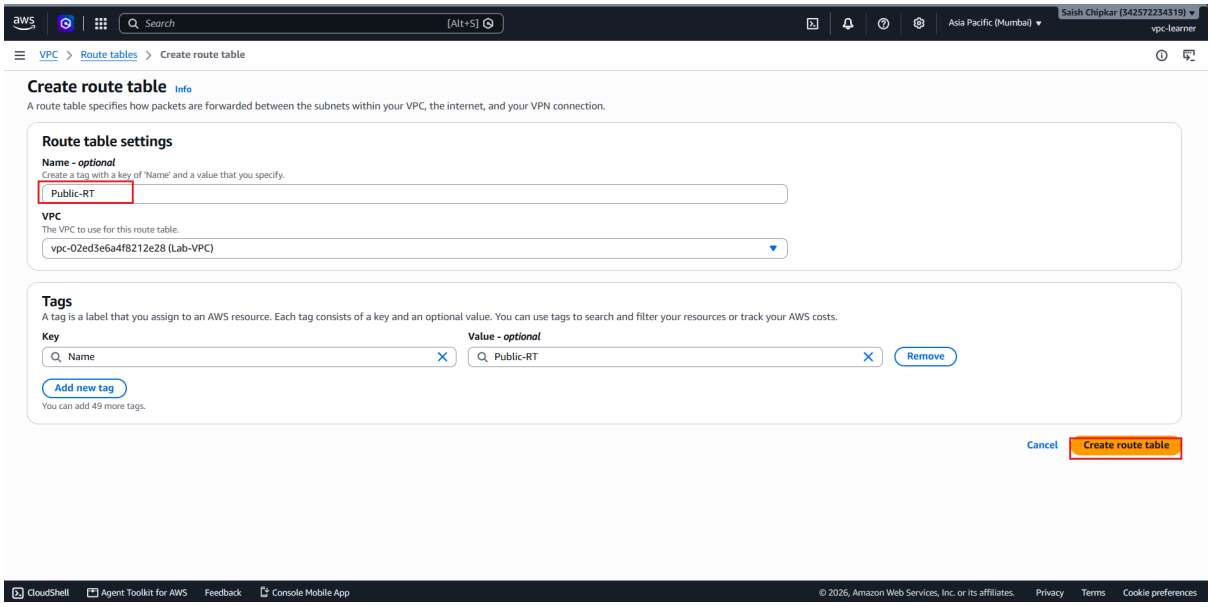


Figure 28: Creating the Public Route Table

Once created, navigate to the **Routes** tab and edit the routes. Add a route with the destination `0.0.0.0/0` targeting the previously created Internet Gateway (`igw-...`)[cite: 18].

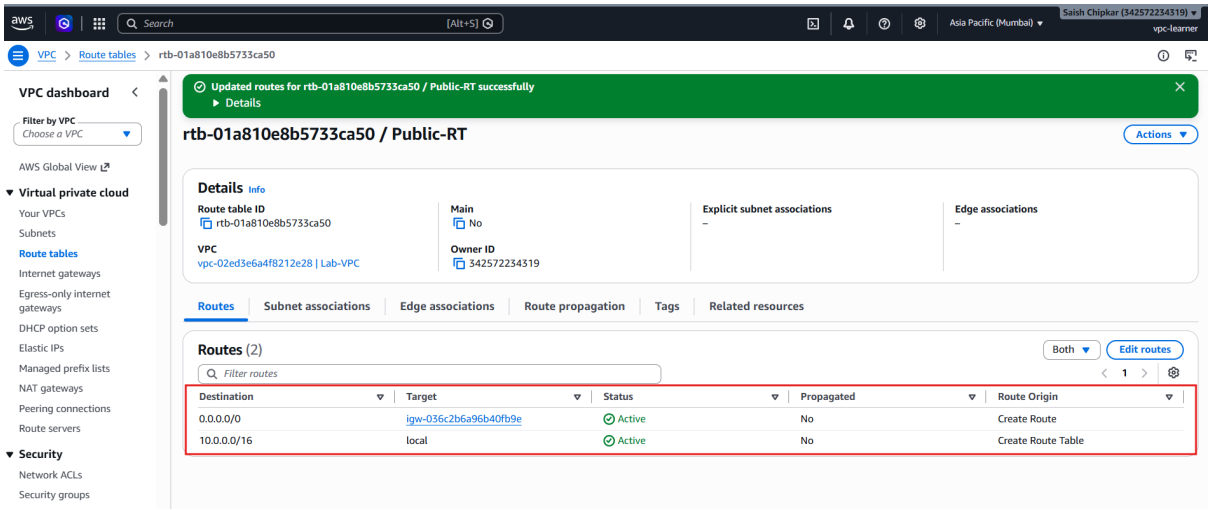


Figure 29: Adding IGW route to Public-RT

Navigate to the **Subnet associations** tab, select the `Public-Subnet`, and click **Save associations**[cite: 19].

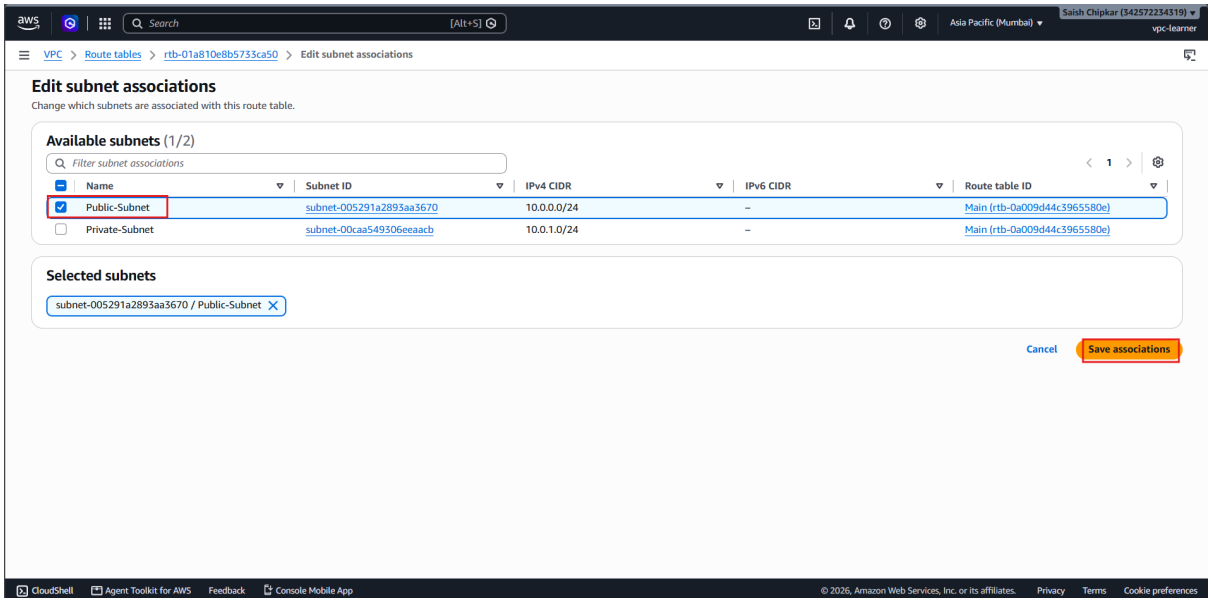


Figure 30: Associating Public-Subnet with Public-RT

10.2 Private Route Table

Create a second route table named `Private-RT` for the private subnet, associating it with `Lab-VPC`[cite: 20]. (Subsequent steps will involve routing `0.0.0.0/0` for this table to the NAT Gateway).

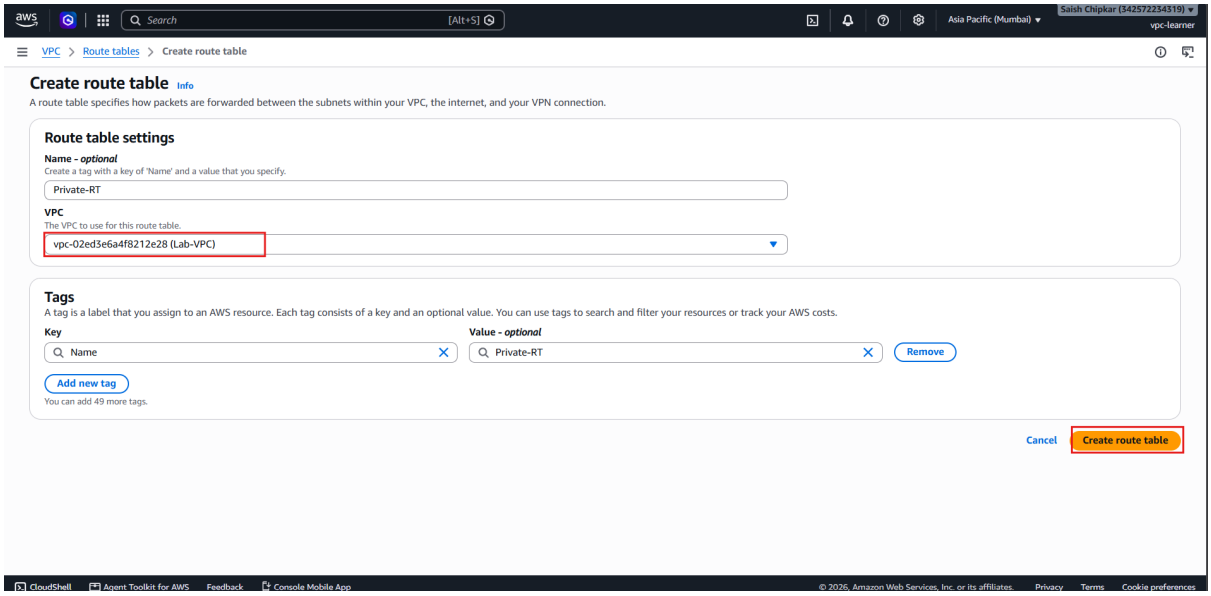


Figure 31: Creating the Private Route Table

10.3 Completing the Private Route Table

Navigate back to the **Private-RT** route table. Under the **Routes** tab, edit the routes to add a new destination for `0.0.0.0/0`. Set the target to the newly created NAT Gateway (`nat-...`) and save the changes.

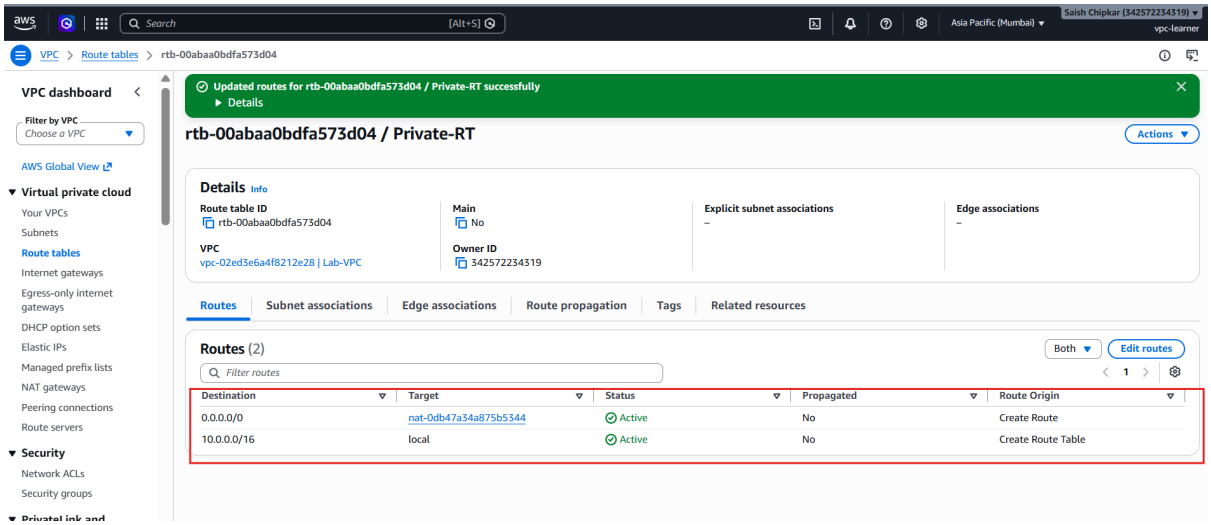


Figure 32: Adding the NAT Gateway route to Private-RT

Next, switch to the **Subnet associations** tab for the Private Route Table. Select the `Private-Subnet` and click **Save associations**. Instances in the private subnet can now route outbound internet traffic through the NAT Gateway.

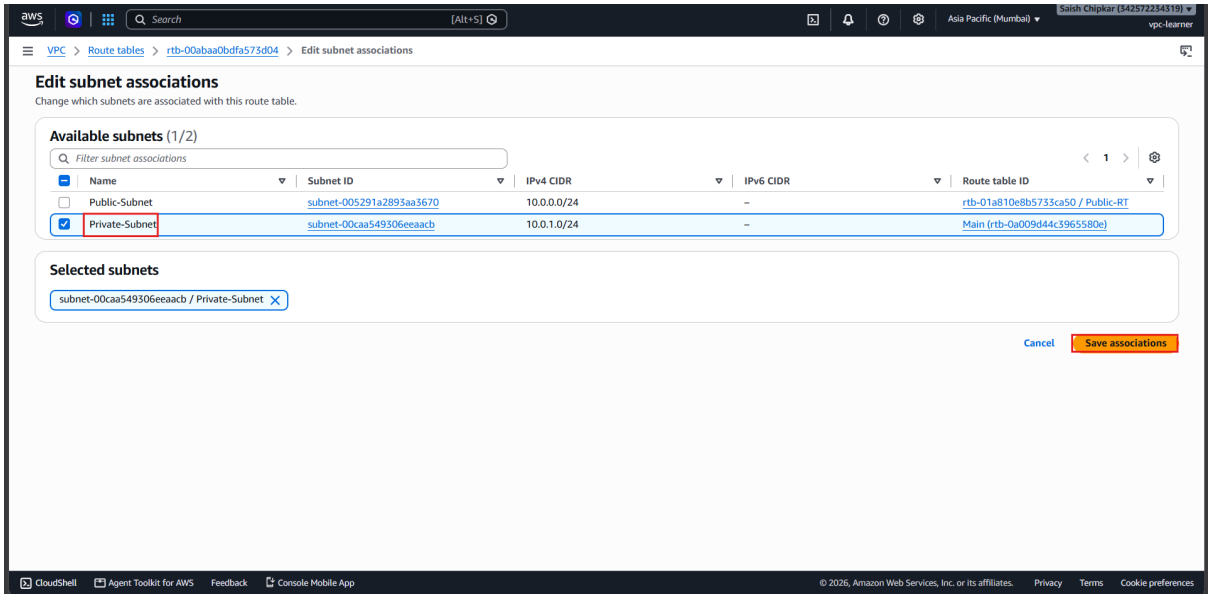


Figure 33: Associating Private-Subnet with Private-RT

11 Deploying EC2 Instances for Testing

11.1 Launching the Public Instance (Bastion Host)

Navigate to the EC2 Dashboard and launch a new instance. Configure the network settings to place the instance in the `Public-Subnet` of `Lab-VPC`. Ensure that **Auto-assign public IP** is **Enabled**. Create or select an existing security group allowing inbound SSH (port 22) traffic.

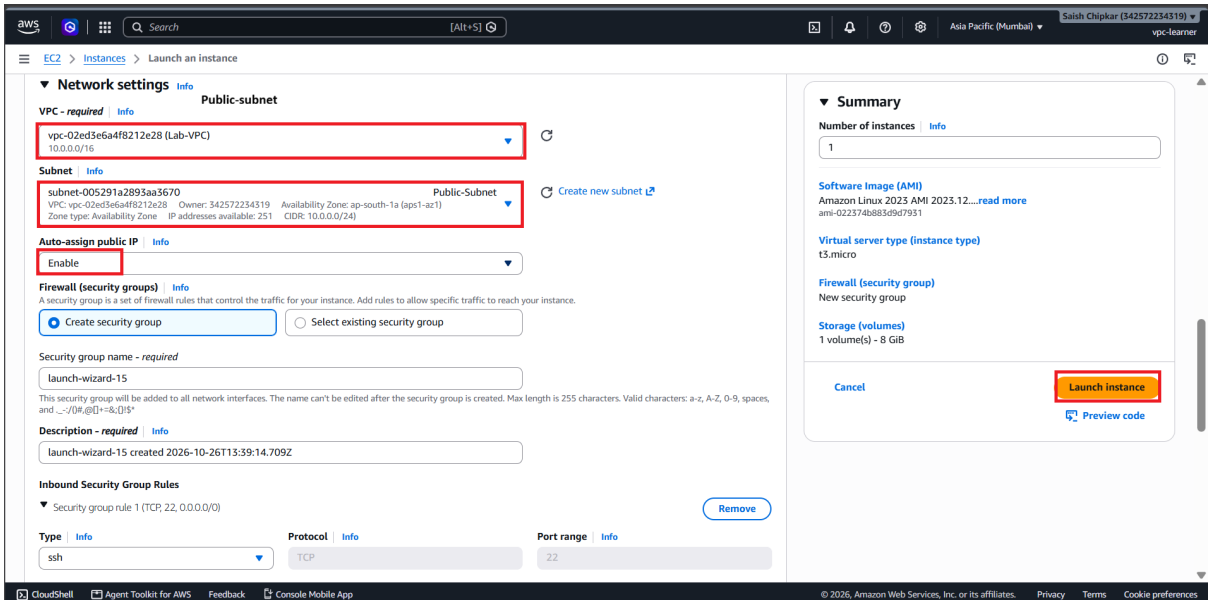


Figure 34: Launching the Public EC2 Instance

11.2 Launching the Private Instance

Launch a second instance, placing it in the `Private-Subnet`. Ensure that **Auto-assign public IP** is **Disabled**, as this instance should only be accessible internally. Select a key pair (e.g., `privatekey`) that will be used to access this instance.

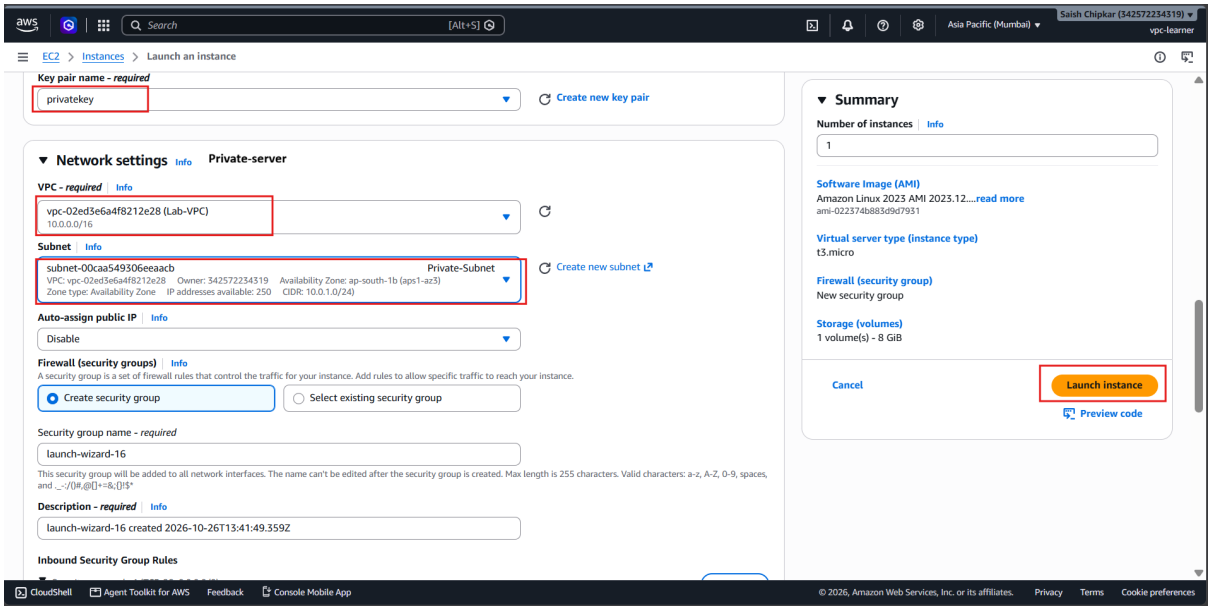


Figure 35: Launching the Private EC2 Instance

12 Testing Connectivity and Key Transfer

12.1 Connecting to the Public Instance

Using a terminal or command prompt, SSH into the public instance using its public IP address and the associated private key (e.g., `key18.pem`).

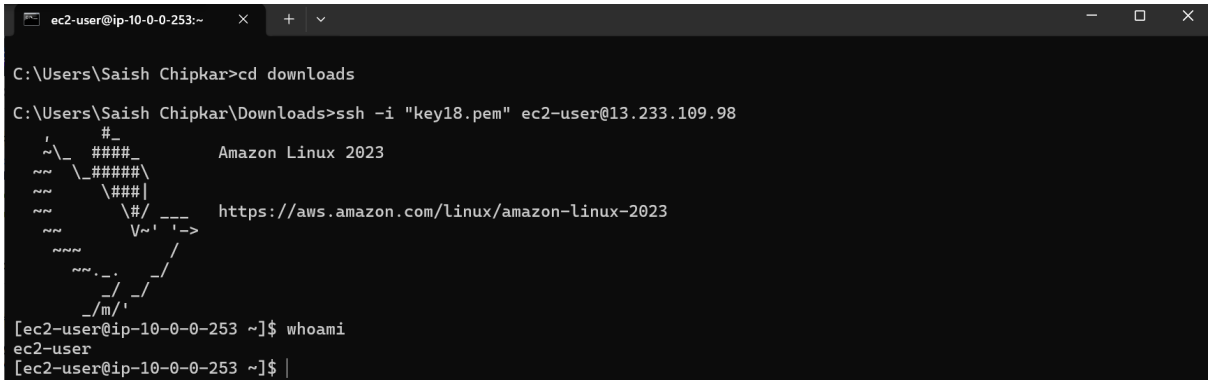


Figure 36: SSH into the Public Instance

12.2 Transferring the Private Key via SCP

To SSH from the public instance into the private instance, the private key must be transferred to the public server. From the local machine, use the Secure Copy Protocol (`scp`) to securely transfer the `privatekey.pem` file to the `/home/ec2-user/` directory of the public instance.

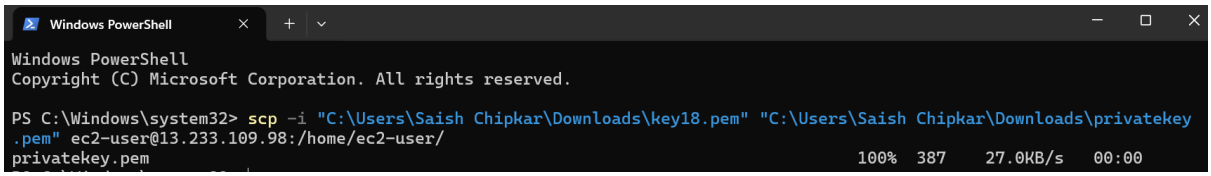


Figure 37: Transferring the private key using PowerShell and SCP

12.3 Configuring Key Permissions

Once the key is transferred, log back into the public instance and verify its presence. For the SSH client to accept the key, its permissions must be restricted. In this experiment, the user switches to the root account (`sudo su`), changes the file ownership to `root:root`, and restricts permissions using `chmod 700`.

Note: The standard AWS best practice for SSH keys is to leave ownership with `ec2-user` *and set permissions to read-only using* `chmod 400`*, though both methods ensure the key is protected from open access.*

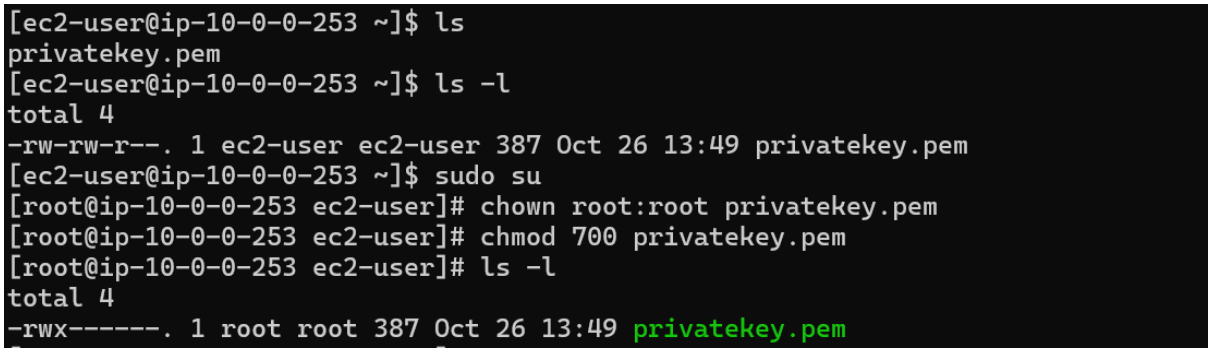


Figure 38: Adjusting permissions on the transferred private key

13 Final Verification

After configuring the necessary key permissions on the bastion host, we can proceed to access the private instance and verify its outbound connectivity.

13.1 Accessing the Private Instance

From the public bastion host, initiate an SSH connection to the private instance using its internal IP address. Ensure the transferred private key is referenced in the command. You will be prompted to verify the host's authenticity upon the first connection; type `yes` to proceed[cite: 30].

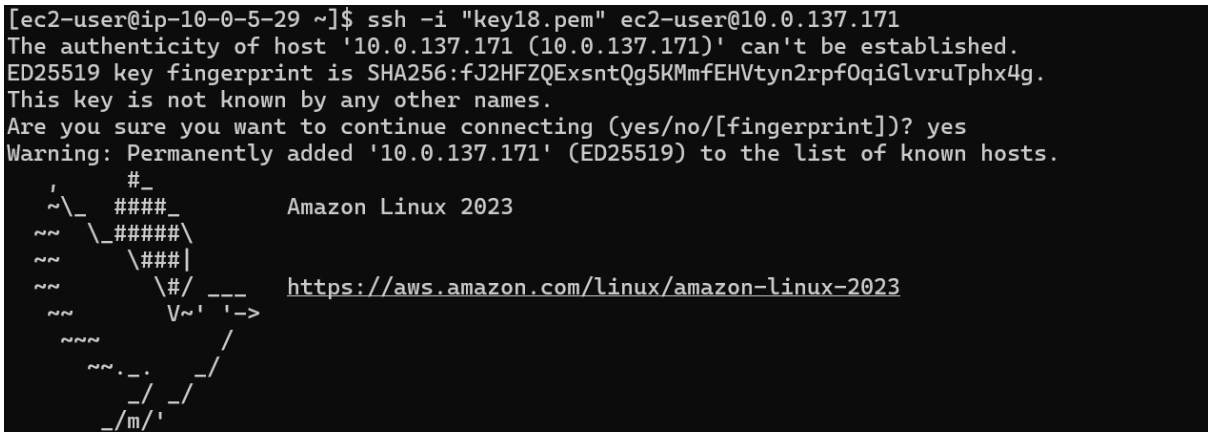


Figure 39: SSH connection established from the bastion host to the private instance

13.2 Testing Outbound Internet Connectivity

Once logged into the private instance, execute a standard `ping` command targeting a known external IP address (e.g., Google's public DNS server at `8.8.8.8`) to test ICMP routing.

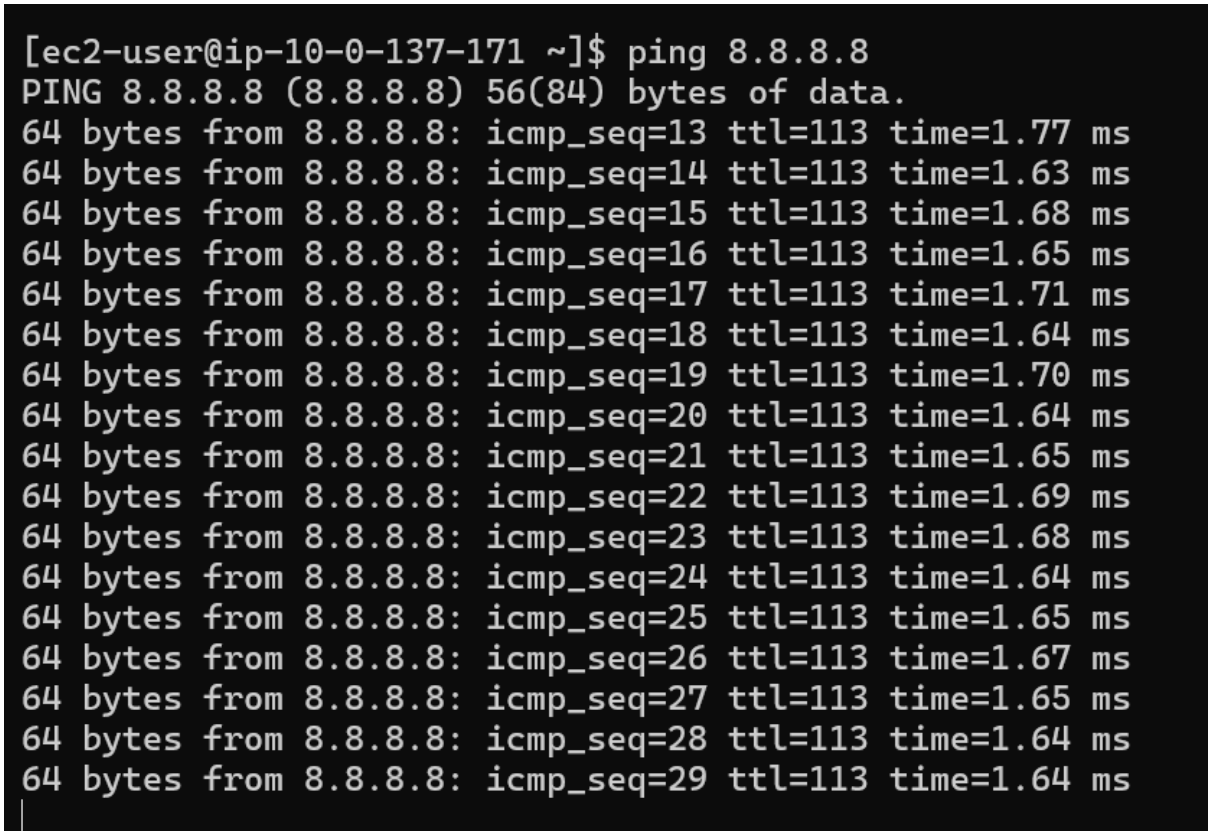


Figure 40: Successful ICMP ping from the private instance to 8.8.8.8

The successful transmission and reception of packets confirms that the private instance can reach the external internet[cite: 29]. Because the instance resides in a private subnet without a public IP, this successful connection proves that the Zonal NAT Gateway is correctly performing Network Address Translation and that the Private Route Table is accurately routing `0.0.0.0/0` traffic through the gateway.

14 Experimental Results and Summary

The objective of this laboratory experiment was to design, configure, and verify a secure, two-tier network architecture within Amazon Web Services (AWS). This objective was successfully achieved, demonstrating the practical implementation of core cloud networking components.

14.1 Infrastructure Configuration Summary

A custom Virtual Private Cloud (VPC) was established and segmented into two distinct environments:

- **Public Tier:** A public subnet configured with a route table directing default outbound traffic (`0.0.0.0/0`) to an Internet Gateway (IGW).
- **Private Tier:** A private subnet configured with a route table directing default outbound traffic to a Zonal NAT Gateway.

The NAT Gateway was deployed within the public subnet and assigned an Elastic IP address to facilitate Network Address Translation for the private resources.

14.2 Verification of Secure Access and Routing

The network's isolation and routing policies were validated by deploying Amazon Linux 2023 EC2 instances into both subnets:

1. **Inbound Isolation:** Direct external access to the private instance was successfully blocked. Access was only achieved by utilizing the public EC2 instance as a bastion host (jump server) and securely transferring the SSH private key via the Secure Copy Protocol (SCP).

2. **Outbound Connectivity:** From within the isolated private instance, external ICMP traffic testing (`ping 8.8.8.8`) was executed successfully.

14.3 Conclusion

The successful `ping` execution from the private subnet serves as definitive proof that the NAT Gateway and associated Route Tables are functioning correctly. The NAT Gateway accurately translated the private instance's internal IP address to the public Elastic IP, forwarded the request to the external internet via the IGW, and routed the response back to the private server.

Ultimately, this experiment successfully produced an industry-standard AWS topology that shields internal workloads from inbound internet threats while maintaining the necessary outbound connectivity for external software updates and API communications.